

01 Quick Reference: "Which File Do I Edit?"

This table maps every system feature to its runtime file on the target board, its persistent source location in the Yocto layer repository, and the systemd restart command:

Feature / Component	On Board (Runtime Path)	Repository Source (Yocto Recipe)	Service Restart
Ethernet eth0 (Port 1)	/etc/systemd/network/10-eth0.network	meta-custom-hmi/recipes-core/systemd/files/10-eth0.network	systemctl restart systemd-networkd
Ethernet eth1 (Port 2)	/etc/systemd/network/11-eth1.network	meta-custom-hmi/recipes-core/systemd/files/11-eth1.network	systemctl restart systemd-networkd
OTA Server URL & Interval	/etc/ota-server.conf	meta-custom-hmi/recipes-core/ota-agent/files/ota-server.conf	systemctl restart ota-agent
Display Sleep Timeout	/etc/hmi-sleep.conf	meta-custom-hmi/recipes-hmi/hmi-sleep/files/hmi-sleep.conf	systemctl restart hmi-sleep
USB Auto-Update Script	/usr/bin/qt-auto-launcher.sh	meta-custom-hmi/recipes-hmi/qt-auto-launcher/files/qt-auto-launcher.sh	systemctl restart qt-auto-launcher
Default Bundled Executable	/opt/hmi/bin/app	meta-custom-hmi/recipes-hmi/qt-auto-launcher/files/test	systemctl restart qt-auto-launcher
Device Tree (Pads & Buses)	/boot/okmx6ull-c-emmc.dtb	meta-custom-hmi/recipes-kernel/linux/files/imx6ull-custom-hmi.dts	Rebuild kernel / reboot

02 GPIO Control & P17 Expansion Header

Linux calculates sysfs GPIO numbers on i.MX6ULL via: $\text{GPIO Number} = (\text{Bank} - 1) \times 32 + \text{Pin}$.

- GPIO1 = gpiochip0 (Pins 0–31) • GPIO2 = gpiochip1 (Pins 32–63) • GPIO3 = gpiochip2 (Pins 64–95)
- GPIO4 = gpiochip3 (Pins 96–127) • GPIO5 = gpiochip4 (Pins 128–159)

P17 Pin	SoC Pad	Linux Sysfs GPIO	gpio Line	Default Mode	Notes & Safety
Pin 8	CSI_HSYNC	GPIO 116 (GPIO4_I020)	gpiochip3 Line 20	User GPIO / I2C2 SCL	Primary user GPIO pin. Fully safe for arbitrary digital I/O.
Pin 9	CSI_MCLK	GPIO 113 (GPIO4_I017)	gpiochip3 Line 17	User GPIO / I2C1 SDA	Shared with Touchscreen and RTC on I2C1 bus.
Backlight	GPIO1_I008	GPIO 8 (GPIO1_I008)	gpiochip0 Line 8	PWM / Backlight	LCD backlight brightness & display sleep control.
UART3 TX	UART3_TX	GPIO 24 (GPIO1_I024)	gpiochip0 Line 24	UART3 Transmit	Repurposable if UART3 peripheral is unused.
UART3 RX	UART3_RX	GPIO 25 (GPIO1_I025)	gpiochip0 Line 25	UART3 Receive	Repurposable if UART3 peripheral is unused.

Controlling GPIO Lines (Command Line)

```
# Method 1: Using modern libgpiod tools (Recommended)
gpioset gpiochip3 20=1 # Drive Pin 8 (chip 3 line 20) HIGH
gpioset gpiochip3 20=0 # Drive Pin 8 LOW
gpioget gpiochip3 20 # Read current logic state (returns 0 or 1)

# Method 2: Using legacy sysfs interface (/sys/class/gpio)
echo 116 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio116/direction
echo 1 > /sys/class/gpio/gpio116/value
cat /sys/class/gpio/gpio116/value
echo 116 > /sys/class/gpio/unexport
```

03 UART Serial Interfaces

Device Node	Hardware Port	Default Baud	Role & Description
/dev/ttymx0	UART1	115200 8N1	Debug Serial Console (Connected to onboard USB-to-UART bridge / host PC)
/dev/ttymx1	UART2	Application default	General Purpose RS232 / TTL peripheral interface
/dev/ttymx2	UART3	Application default	General Purpose TTL interface (P17 expansion pins)
/dev/ttymx3	UART4	Application default	General Purpose RS485 / TTL peripheral interface
/dev/ttymx4	UART5	Application default	General Purpose TTL peripheral interface

Testing & Communicating via UART:

```
# Configure baud rate and raw transmission mode
stty -F /dev/ttymx1 115200 raw -echo

# Transmit data frame
echo "TEST_DATA_FRAME" > /dev/ttymx1

# Read incoming stream
cat /dev/ttymx1
```

04 SPI Bus Interfaces & Loopback Verification

Device Node	Controller	Max Clock Speed	Typical Peripherals
/dev/spidev0.0	ECSPI1, CS0	Up to 20 MHz	SPI Flash, EEPROM, DAC/ADC, external sensors
/dev/spidev1.0	ECSPI2, CS0	Up to 20 MHz	Secondary SPI bus expansion

Hardware Loopback Verification (MOSI ↔ MISO):

Short the MOSI and MISO pins together on the board header and execute:

```
# Transmit 16-byte reference frame at 1 MHz clock frequency
spidev_test -D /dev/spidev0.0 -s 1000000 -v
```

05 Dual Ethernet Configuration (systemd-networkd)

The system uses systemd-networkd with dedicated per-interface configuration files:

- Port 1 (eth0 / fec1): /etc/systemd/network/10-eth0.network
- Port 2 (eth1 / fec2): /etc/systemd/network/11-eth1.network

Profile A: Static IP Address	Profile B: Auto-IP (Link-Local) + DHCP (Default)
<pre>[Match] Name=eth0 [Network] Address=192.168.1.150/24 Gateway=192.168.1.1 DNS=8.8.8.8 DNS=1.1.1.1 LinkLocalAddressing=no IPv6AcceptRA=no</pre>	<pre>[Match] Name=eth0 [Network] DHCP=yes LinkLocalAddressing=ipv4 MulticastDNS=yes LLMNR=yes IPv6AcceptRA=no [DHCPv4] RouteMetric=10</pre>

Applying Network Changes:

```
systemctl restart systemd-networkd
networkctl status eth0
ip addr show eth0
```

System Services: OTA, Sleep & USB Update

Firmware Upgrades, Power Management & Zero-Network Application Deployment

06 Remote OTA Update Agent Service

The `ota-agent.service` daemon runs in the background, polling an HTTP server for firmware update packages (`update.swu`) and writing them into the alternate rootfs bank (Dual A/B Bank fail-safe architecture with automated rollback).

Setting	Configuration File	Default Value	Description
OTA_SERVER_URL	/etc/ota-server.conf	http://192.168.1.100:8000	Base HTTP endpoint hosting <code>version.json</code> & <code>update.swu</code>
CHECK_INTERVAL	/etc/ota-server.conf	30	Background check frequency in seconds

```
# Modify server URL or interval, restart service, and inspect live activity:
vi /etc/ota-server.conf
systemctl restart ota-agent
journalctl -u ota-agent -f
```

07 Display Sleep & Touch-to-Wake Service

The `hmi-sleep.service` daemon monitors `/dev/input/touchscreen0`. When no touch events occur for the configured inactivity period, it blanks the LCD framebuffer and turns off the backlight power. Touching anywhere on the screen immediately wakes the display.

Setting	Configuration File	Default	Supported Values
SLEEP_TIMEOUT_SEC	/etc/hmi-sleep.conf	120 (2 min)	0 = Always ON (Sleep disabled) • 30 = 30s • 300 = 5 min

```
# Change sleep timeout and apply immediately:
sed -i 's/^SLEEP_TIMEOUT_SEC=.*$/SLEEP_TIMEOUT_SEC=300/' /etc/hmi-sleep.conf
systemctl restart hmi-sleep
```

08 USB Flash Drive Auto-Update Workflow

The board's launcher service (`qt-auto-launcher.service`) automatically checks attached USB flash drives on boot or restart, updates the application binary at `/opt/hmi/bin/app`, and executes it immediately.

Format Compatibility: Format the USB flash drive as **FAT32**. The launcher automatically mounts and handles both partitioned drives (`sda1`) and whole-disk drives (`sda`).

Step 1: Prepare the USB Flash Drive on Any PC

Copy these **two files** directly into the **root directory** of the FAT32 USB drive:

Filename	Type	File Content / Purpose
UpdateCont	Text File	Contains single character: <code>Y</code> (authorizes auto-update)
test or app	ARM 32-bit Executable	New compiled Qt ARM application binary to install

Step 2: Trigger Update on the Board

- Method A (Power-On / Reboot):** Insert the USB drive and reboot the board. The update applies automatically on boot.
- Method B (Live Update without Reboot):** Insert the USB drive and run via console:

```
systemctl restart qt-auto-launcher
```

Step 3: Verification

```
# 1. View launcher update log
journalctl -u qt-auto-launcher -n 25 --no-pager

# 2. Confirm binary size and timestamp
ls -l /opt/hmi/bin/app
```